Grounding generative AI in the real world and objective reasoning

List of authors

- Pedro Fernandes - primeiro autor
- Lucas Guerra - primeiro autor
- Philippos Costa
- Nome a ser indicado pela NAIA
- Thiago Daibes Padilha - autor sênior e encarregado da submissão

Introduction

The Physical Grounding Hypothesis, the Physical Symbol System Hypothesis and Neurosymbolic AI aren't new ideas. Adherents of the Physical Symbol System Hypothesis include Herbert Simon and Allen Newell. In 1976, when they won the Turing Award, they said in their acceptance lecture:

> A physical symbol system has the necessary and sufficient means for general intelligent action.

Readers of the present paper may be asking the kind of circular question Warren McCulloch [2] made famous: What is a symbol that intelligence may use, and intelligence that it may use a symbol? The authors leave it to them to analyze Newell and Simon's Turing Lecture and determine what the Turing Award laureates mean by physical symbols. They argue that there is no "intelligence principle." But the lack of a simple *deus ex machina* does not imply that there are no structural requirements for intelligence. One such requirement is the ability to store and manipulate symbols.

Computer scientists and neuroscientists who adopted the PSSH soon run into a difficulty. However clever the scientists designing and implementing a physical symbol system, they would die before adding all the details present in the real world, society and language to their model. And they were indeed clever: Herbert Simon won the Nobel Memorial Prize, and Joshua Lederberg won the Nobel Prize in Medicine. Marvin Minsky won the Turing Award.

In other words, some designs are so complex that people can't implement them in a lifetime. Language has so many rules, and they change so fast, that AI researchers

struggle to build a grammar checker or translator [3]. Common sense has so many rules that CyC researchers couldn't implement it in 30 years [4][5].

John McCarthy discovered the enormous complexity of PSSH very early in the history of AI research [6]: "The qualification problem … arises when we try to express the conditions for actions. We can think of the robot as having a general rule that it can fly to Moscow if it has a ticket. But suppose the ticket turns out to be counterfeit, the airport is closed by a strike, or the robot has been disassembled. To express all these qualifications in the antecedent of a conditional would make the rule unmanageably large, and we could never be sure we had thought of everything."

In the heroic times of Artificial Intelligence, researchers worked with both symbolic systems and neural networks. Marvin Minsky, whom many consider the paragon of PSSH, wrote a thesis on neural networks [7]. In 1951, he co-built SNARC (Stochastic Neural Analog Reinforcement Calculator), which is widely considered the world's first artificial self-learning neural network machine. [8,9].

With the rise of Generative AI after 2017, and the publication of a paper on the Transformer architecture [10], people began to notice that neural networks had problems complementary to PSSH.

Neurosymbolic AI, which found a strong champion in Gary Marcus [11], proposes a hybrid system: generative models provide imense detail, while PSSH provides trustworthiness.

The Physical Grounding Hypothesis was proposed in 1990 by Rodney Brooks[12]: "Nouvelle AI is based on the physical grounding hypothesis. This hypothesis states that to build an intelligent system, it is necessary to ground its representations in the physical world." We left out the second part of Rodney Brooks, because we don't believe in it: "Our experience with this approach is that once this commitment is made, the need for traditional symbolic representations soon fades entirely."

Physical grounding

We will start this section by making it clear that there isn't such a thing as an information processing system that is absolutely trustworthy. We can reduce hallucination in a neurosymbolic AI, but a residual margin for spluttering, juddering and failure will always remain.

We were inspired by the PERCEPT platform for grounding a Generative AI in the physical world [13,14, 15].

To train a generative AI for single-cell RNA-seq, members of Jeffrey Ishizuka's laboratory use the Cell2Sentence algorithm to convert sequencing data into text. Growing cancer cells in vitro allows researchers to test drugs. However, Perry and colleagues explain that many immunotherapies with promising preclinical results fail in clinical trials[13], partly because experimental models do not adequately represent human tumor–immune interactions. In PERCEPT, they therefore prepare cultures containing tumor and immune cells from patient samples. They expose portions of these cultures to treatments and leave others unstimulated as controls.

Single-cell RNA sequencing measures gene expression in thousands of cells from the treated and control cultures. The expression "single-cell" means that each cell's RNA is measured separately, allowing researchers to distinguish individual cells and different cell types within the culture. Suppose a treated cell expresses more of a gene than a control cell. That difference might come from the drug, or the two cells might already differ in cell type or state. To determine what changed, researchers would ideally measure the same cell before and after treatment. Sequencing, however, destroys the cell being measured.

Jeffrey Ishizuka's collaborators use CINEMA-OT to separate gene expression patterns correlated with treatment from patterns reflecting differences between the cells themselves. They use the latter to match treated and untreated cells with similar characteristics, such as cell type and cell-cycle stage.

Each match estimates how a treated cell might have behaved without the drug. These counterfactual matches allow researchers to compare gene-expression profiles and estimate treatment effects. This resembles comparing treatment and control groups in a clinical trial, although the cell matches are computed rather than established by random assignment [13,15].

The authors developed a system to minimize hallucinations from medical Generative AI for a limited kind of queries. Physicians often want answers restricted to a well-defined set of papers. These papers, which can be downloaded as PDF, DOCX, and other formats, are converted to Markdown text. Biomedical survival and response plots are represented as sexpr, the traditional PSSH data structure. Diagnostic images (MRI, CT scans, etc.) are also represented as sexpr. Generative AI handles these textual data representations much better than the original images

and plots. If a human is reading the paper, a command in a special editor renders images or plots back into a form similar to the original publication.

When the query is submitted to the medical AI, the documentation is pasted together. We could say the medical AI is doing an open-book test, such as ABIM Maintenance of Certification. In their benchmarks, the authors always submitted the questionnaire together with the papers. However, we can envision physicians subscribing to an AI service with a library of textfyed articles, guidelines and laboratory data, such as single-cell RNA-seqs.

We must answer a question to appraise the approach described here. Do clinicians often consult specific literature to solve problems? Research on clinical information-seeking behavior shows that doctors generate multiple questions per day and pursue answers to about half of them; when they do, they favor summarized, synthesized resources rather than searching databases for individual journal articles [16, 17, 18]. Medical AI, such as NAIA and OpenEvidence, tries to provide such synthesized resources.

Objective reasoning

Plato is one of the first thinkers to propose logic and mathematics to verify one's statements. For instance, he claims that calculation (τὸ λογίζεσθαί τε καὶ ἀριθμεῖν) is one of those methods that lead toward the truth (τῶν πρὸς τὴν ἀλήθειαν ἀγόντων ἂν εἶναι τοῦτο τὸ μάθημα) [19]. On another occasion, the philosopher says that geometrical knowledge could draw the soul toward truth (τοῦ γὰρ ἀεὶ ὄντος ἡ γεωμετρικὴ γνῶσίς ἐστιν. — ἑλκτικόν, ᾧ γενναῖε, ψυχῆς πρὸς ἀλήθειαν) [20].

The belief that opinion and thought must be checked against logic and mathematics forms the conceptual backbone of Plato's philosophy, and has had a profound impact on modern science and technology. After all, it is less expensive and faster for an aerospace engineer to run simulations and finite element calculations than to build a prototype airplane.

Urbain Le Verrier's discovery of Neptune is legendary. The French astronomer calculated the unknown planet's predicted position by analyzing gravitational irregularities (perturbations) in Uranus's orbit. He sent his calculations to the Berlin Observatory. On September 23, 1846, Johann Gottfried Galle & Heinrich Louis d'Arrest received Le Verrier's letter at the Berlin Observatory, and they visually identified Neptune that very evening within one degree of the predicted spot using a telescope and star charts.

Contemporary civilization added computers to Le Verrier and Plato's logic, calculations and mathematics. Computers brought the concept of a formal system into the spotlight. Mathematics, grammar production rules, predicate calculus (as in the computer language Prolog) and algorithms are examples of formal systems. Computers can execute algorithms, predicate calculus and many other formal systems. Formal systems must have the following metatheoretical requirements:

1. Precision & Unambiguity that eliminate human bias and natural-language misinterpretation.
2. Consistency. The system cannot derive both a statement and its absolute contradiction.
3. Determinism & Reproducibility. Given identical initial states (inputs) and operations, the system must yield identical results (outputs).
4. Conceptual Minimalism. No axiom in the system can be derived or proven using the other axioms.
5. Decidability. There must be an algorithmic, finite process to check whether a given string is a valid formula or axiom.

Computers traditionally offer services that meet many, if not all, requirements of a formal system (usually, unambiguity, consistency and reproducibility):

- Operating Systems (Linux, Windows). While the underlying hardware is chaotic, the service an OS provides is completely unambiguous.
- Compilers & Statistical Packages (GCC, R). They are entirely rule-governed. Input the same code or dataset, and they predictably emit the same binary or mathematical model.
- Extensible Text Editors (Emacs, Neovim): Their configuration languages (Emacs Lisp, Lua) are strict formal environments. They process text via precise, deterministic commands and regular expressions.
- Electronic Medical Records (Epic). They use a strict relational database schema and standardized data fields (like ICD-10 medical codes). The input format is unambiguous, ensuring data consistency across clinics.
- CT Scans & GPS. Their core mathematical algorithms (the Radon transform for CT; trilateration equations for GPS) use strict geometric and physical formulas to reproduce an image or a coordinate.
- Cryptographic & Blockchain Services (e.g., Bitcoin): These are the ultimate modern examples. They execute transactions based on entirely transparent, deterministic rules.

People were used to the paradise of Cantor and Hilbert's formal systems. Then generative AIs appeared that don't obey unambiguity, consistency and reproducibility. They share this unruly behavior (trait?) with humans, and it is exactly this anthropomorphism that makes them interesting. In any case, physicians and lawyers keep treating Generative AI as if it automatically provides the checks and balances of formal systems. They don't. Therefore, the present paper aims to tighten the running clearance and minimize the radial play of Generative AI within the bearings of deterministic system services.

We already have a successful example of reduction bushing and deterministic guardrailing for AI. Codex and Claude check the programs they generate by running them in virtual machines. In the next section, Claude will describe exactly how it helps us with Schememax and RASP (Robust Accurate Statistical Parsing).

Human appraisal of Schememax

Schememax is a text editor connected with Generative AI assistants. It is compatible with GNU Emacs, a Lisp dialect (Elisp) optimized for text manipulation. While users interact with it as an extensible editor, the interface is just an application running on top of a C-based Elisp interpreter. Beyond the editor, Emacs offers many services, including a massive built-in system called GNU Emacs Calc. Calc is an incredibly advanced RPN and algebraic calculator capable of symbolic computation, matrix operations, and calculus. Emacs also offers a platform-agnostic command-line shell called eshell. Because common utilities (like ls, cp, and mkdir) are rewritten directly in Elisp, Eshell behaves identically across any operating system Emacs runs on, including Windows, Linux and macOS.

The difference between Emacs and Schememax is that Emacs provides primarily quasi-formal systems, while Schememax provides Generative AI services. One of the authors of the present paper developed the core of Schememax with help from Hugh Barney. Besides the basic coder editor resources written in C, this core includes an integrated Lisp virtual machine (femtoscheme) that Generative AI can manipulate safely (we hope) through special bridges to Claude and ChatGPT. This lets Anthropic's Claude fix bugs and add new commands to Schememax. Then, while Emacs is extended by more than 10,000 people who collaborated with 10+ million lines of heavily peer-reviewed code, Schememax is entirely debugged and extended by Generative AI, such as Claude Code, as we will see in the next section.

Human appraisal of RASP

The Physical Symbol System Hypothesis (PSSH) promises predictable, unambiguous, consistent and hallucination-free services. People like unerring computing tools, like calculators and Python scripts. The problem with PSSH is that human developers who build these AI tools are often overwhelmed by the real-world details and give up on their projects. This was the case for many Natural Language Processing tools, such as translators, summarizers and grammar checkers. The authors of the present paper believe that Generative AI can safely add value to the endeavors of the brilliant scientists who created imaginative tools under the aegis of PSSH. They test this conjecture by adding rules to RASP so it can parse a significant subset of English.

Claude's grounding in quasi-formal systems

Date of this answer: Tuesday, October 20, 2026.

Where the programs ran

In this session, almost every check ran in a temporary Linux virtual machine on a Mac, the one Claude's desktop app creates. Only one step ran in a temporary Linux container in Anthropic's cloud: compiling irene-dixe and drawing the test table, because the virtual machine had no SDL.

RASP

RASP stands for Robust Accurate Statistical Parsing. That was the name of the UK EPSRC project (grants GR/N36462 and GR/N36493) under which Ted Briscoe, John Carroll and colleagues built the parser at Cambridge and Sussex. Reference [3], Briscoe and Carroll's LREC 2002 paper, has a longer title: "Robust Accurate Statistical Annotation of General Text." sbcl-rasp is RASP 3.3a ported to SBCL and repaired.

The method: the model proposes, the machine decides

A language model cannot be trusted to judge its own fix: it produces plausible code and plausible claims with equal fluency. So no fix counts until a deterministic program has run on the changed code and given the expected result. The cycle has five steps.

The first step is to reproduce the error before fixing it: the old program must show the fault on a concrete input. For example, the old Schememax build, sent C-s, "Ishizuka" and the up arrow, turned the text into "Jeffrey Ishizuka[A for help." This proves that the diagnosis points at the real cause, not a guessed one. The second step is to change the code, which is the model's conjecture. The third step is to build and run the program on the same input; the new build moved the cursor up one line and left the buffer unmodified.

The fourth step compares the result against an oracle, which is either the old build (differential testing) or a reference output. In Schememax, the up arrow at the end of a word-wrapped file stayed at position 216 in the old build, while the fixed build went from 216 to 156 and then to 96. A file ending with a newline gave 192 in both builds, confirming that the case meant to stay unchanged did stay unchanged. In RASP, the oracle is the original parser's output under Clozure CL on the 560 DepBank sentences, and every repair must leave those analyses identical, with 0 differing lines. Then come the corpora: 81 correct sentences written by Codex (68 parsed by RASP 3.3a, 81 after the repairs) and 27 known-bad sentences (17 rejected before, 25 after).

The fifth step runs the whole regression suite: for Schememax, 222 checks in a headless build plus femtoscheme's 137 tests; for RASP, `make check`. A fix that breaks something elsewhere fails here.

For RASP, a new rule (a lexicon entry, a tokeniser or sentence-splitter change) follows the same cycle. A sentence the rule should fix must now parse, the known-bad sentences must still be rejected, and DepBank must still match the oracle. The CHANGES file records each repair with the sentences it fixes. For example, the tokeniser split "1990" into "1 9 9 0", and the Clozure oracle showed the same split. That located the fault in the original build, not in the port.

What the deterministic check caught in this session

These errors are worth reporting, because they show the guardrail doing work rather than ceremony. The first was a load-order bug: my assistant.scm called a function that another file defines later. Every isolated test passed, and only the full editor build revealed it ("variable set-buffer-menu has no value"). The second was a pair of help lines that I had made longer than 80 characters, which broke an existing test.

My own test oracles were also wrong three times: a table test expected the wrong string; a test driver dropped spaces, which looked like an editor bug; and an "old build" comparison was invalid because that build already contained the new prompt code. I noticed the last one and discarded that comparison. The checker is only as good as the oracle, and the oracle is also written by someone fallible.

What the check did not cover

The macOS graphical build (SDL) was not run with the new code, and no live request to Claude or Codex was sent through the new prompts. Tests cover the inputs that were tried, nothing more. As Dijkstra put it [21], testing shows the presence of bugs, never their absence. This matches a sentence in grounding-ai.md (the present paper in Markdown): a residual margin of failure always remains. The method shrinks that margin; it does not close it.

Uses of Schememax

Back to human writing – The main application of Schememax is creating ABIM-like Maintenance of Certification tests for medical Generative AI, such as NAIA.

The American Board of Internal Medicine's (ABIM) longitudinal Maintenance of Certification (MOC) is officially called the Longitudinal Knowledge Assessment (LKA) [22]. Introduced as a flexible, lower-stakes alternative to the traditional, high-stakes 10-year proctored recertification exam, it integrates assessment directly into a physician's ongoing routine.

Rules and Testing Environment

- Open-Book Format: The LKA is entirely open-book. Physicians can complete the questions at home or in the office using any online or print resource (such as UpToDate or medical journals), provided they do not consult another person. For medical Generative AI, such as NAIA and OpenEvidence, Schememax uses Claude to create the questions and prepare a textified version of the paper that the medical AI must use to answer them.
- "Life Happens" Allocation: To accommodate busy clinical schedules, physicians may leave up to 100 questions unanswered over the 5-year cycle without penalty. With Schememax, physicians can disallow as many questions as they choose. They disallow a question for different reasons, such as lack of time to review all answers or because it has flaws.

- Immediate Feedback: Unlike traditional exams with delayed results, the LKA acts as a learning tool. After submitting an answer, the physician receives immediate feedback explaining whether the choice was correct, why, and references for further reading. Schememax prepares an answer key containing the solutions and text snippets from the attached paper that justifies the answer. Physicians can use Schememax's answer key and the responses of NAIA and OpenEvidence as a learning tool.

Conclusion

People expect and fully appreciate the unambiguous, consistent and predictable behavior of formal systems. Then, why are they replacing them with unruly Generative AIs? Because formal systems are tough to learn and master. Computer scientists spend years of training before they become fully conversant with the traditional tools of their trade. We hope this paper shows a path to using Generative AI to expand trustworthy traditional computer tools and make them easier to operate.

References

[1] Newell, Allen, and Herbert A. Simon. "Computer Science as Empirical Inquiry: Symbols and Search." Communications of the ACM 19, no. 3 (March 1976): 113–126. https://doi.org/10.1145/360018.360022. Accessed October 19, 2026.

[2] McCulloch, W.S. [1961]. What is a number, that a man may know it, and a man, that he may know a number. General Semantics Bulletin Nos. 26 and 27 (1961), 7–18.

[3] Briscoe, Ted, and John Carroll. "Robust Accurate Statistical Annotation of General Text." In Proceedings of the 3rd International Conference on Language Resources and Evaluation (LREC 2002), 1499–1504, 2002.

[4] Lenat, Douglas B. "CYC: A Large-Scale Investment in Knowledge Infrastructure." Communications of the ACM 38, no. 11 (1995): 33–38.

[5] Lenat, Douglas, Mayank Prakash, and Mary Shepherd. "CYC: Using Common Sense Knowledge to Overcome Brittleness and Knowledge Acquisition Bottlenecks." AI Magazine 6, no. 4 (1985): 65–85.

[6] McCarthy, J. (1977). Epistemological problems of artificial intelligence. In *Proceedings of the 5th International Joint Conference on Artificial Intelligence - Volume 2* (IJCAI'77), 1038–1044.

[7] Minsky, Marvin Lee. Theory of Neural-Analog Reinforcement Systems and Its Application to the Brain-Model Problem. Ph.D. dissertation, Princeton University, 1954. Princeton University Library. Accessed October 19, 2026.

[8] Minsky, M. (1952). *A Neural-Analogue Calculator Based upon a Probability Model of Reinforcement* (Technical Report). Harvard University Psychological Laboratories.

[9] Minsky, M. L. (1954). *Theory of Neural-Analog Reinforcement Systems and Its Application to the Brain-Model Problem* (Doctoral dissertation). Princeton University.

[10] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998–6008.

[11] Lenat, D. B., & Marcus, G. (2023). *Getting from Generative AI to Trustworthy AI: What LLMs might learn from Cyc* (arXiv:2308.04445). arXiv. https://doi.org

[12] Brooks, Rodney A. "Elephants Don't Play Chess." Robotics and Autonomous Systems 6, nos. 1–2 (1990): 3–15.

[13] Perry, C., et al. "Perturbational Single-Cell Profiling of Patient Tumors Defines Lineage- and Context-Specific Programs of Innate Immune Resistance." bioRxiv, preprint, version 1, December 12, 2025. https://doi.org/10.64898/2025.12.09.693245. Accessed October 19, 2026.

[14] Rizvi, Syed Asad, et al. "Scaling Large Language Models for Next-Generation Single-Cell Analysis." bioRxiv, preprint, version 4, January 1, 2026. https://doi.org/10.1101/2025.04.14.648850. Accessed October 19, 2026.

[15] Dong, Mingze, et al. "Causal Identification of Single-Cell Experimental Perturbation Effects with CINEMA-OT." Nature Methods 20 (2023): 1769–1779. https://doi.org/10.1038/s41592-023-02040-5. Accessed October 19, 2026.

[16] Coumou, Herma C. H., and Frans J. Meijman. "How Do Primary Care Physicians Seek Answers to Clinical Questions? A Literature Review." Journal of the Medical Library Association 94, no. 1 (January 2006): 55–60. Full text. Accessed October 20, 2026.

[17] Brassil, Ellen, Bridget Gunn, Anant M. Shenoy, and Rebecca Blanchard. "Unanswered Clinical Questions: A Survey of Specialists and Primary Care Providers." Journal of the Medical Library Association 105, no. 1 (January 2017): 4–11. https://doi.org/10.5195/jmla.2017.101. Accessed October 20, 2026.

[18] Ely, John W., Jerome A. Osheroff, M. Lee Chambliss, Mark H. Ebell, and Marcy E. Rosenbaum. "Answering Physicians' Clinical Questions: Obstacles and Potential Solutions." Journal of the American Medical Informatics Association 12, no. 2 (March–April 2005): 217–224. https://doi.org/10.1197/jamia.M1608. Accessed October 20, 2026.

[19] Plato. Republic. Vol. 2, Books VI–X. Translated by Paul Shorey. Loeb Classical Library 276. Cambridge, MA: Harvard University Press, 1935. Book VII, 525b.

[20] Plato. Republic. Vol. 2, Books VI–X. Translated by Paul Shorey. Loeb Classical Library 276. Cambridge, MA: Harvard University Press, 1935. Book VII, 527b.

[21] Dijkstra, Edsger W. *Notes on Structured Programming*. 2nd ed. Technical Report 70-WSK-03, EWD 249. Eindhoven: Technische Hogeschool Eindhoven, April 1970. [E. W. Dijkstra Archive] (https://www.cs.utexas.edu/~EWD/transcriptions/EWD02xx/EWD249/EWD249.html). Accessed October 20, 2026.

[22] American Board of Internal Medicine. "How Does ABIM Create Assessment Questions?" ABIM Blog, February 1, 2024. https://blog.abim.org/how-does-abim-create-assessment-questions/ Accessed October 21, 2026.